# Rapport Développeur : Implémentation des Algorithmes d'Ordonnancement de Processus

## UE Systèmes d'exploitation

**L3 Informatique**

**Rédigé par**

Tsanta RAZOLIFERASON|Kaiwei Yang|Tenzin Rigsang ZURKHANG

**Table des matières**

UE Systèmes d'exploitation ..... 1
1. Contexte du projet ..... 3
2. Architecture du code et réutilisation des fonctions ..... 3
    2.1 Fonctions de gestion de liste chainée (Liste.c / Liste.h) ..... 3
    2.2 Fonctions de calcul de métriques (Calcul_temps.c / Calcul_temps.h) ..... 3
    2.3 Fonctions d'affichage (ordonnancement.c) ..... 4
    2.4 Fonctions de gestion des processus (ProcStruct.c / ProcStruct.h) ..... 4
    2.5 Module de Lecture de Fichiers de Configuration (lecture_fichier.c/lecture_fichier.h) ..... 4
    2.6 Choix des structures ..... 4
3. Analyse détaillée du fichier ordonnancement.c ..... 5
    3.1 Définitions et constantes ..... 5
    3.2 Fonctions d'affichage ..... 5
    3.3 Fonctions utilitaires ..... 5
    3.4 Fonction principale : ordonnancer() ..... 6
    3.5 Analyse détaillée du fichier main.c (Noyau de l'Interface en Ligne de Commande) ..... 7
4. Points forts du projet ..... 7
    Points forts ..... 7
5. Conclusion ..... 8

# 1. Contexte du projet

Ce projet a pour objectif de simuler l'ordonnancement de processus dans un système d'exploitation, en implémentant quatre algorithmes d'ordonnancement courants :

- **FIFO** (First In First Out)
- **SJF** (Shortest Job First)
- **SJRF** (Shortest Job Remaining First)
- **RR** (Round Robin)

Le simulateur permet de visualiser l'état des processus au fil du temps, de calculer des métriques clés (temps de restitution, temps d'attente, temps de réponse, taux d'occupation du CPU) et de valider le comportement de chaque algorithme.

# 2. Architecture du code et réutilisation des fonctions

Un des points clés du développement est la réutilisation systématique des fonctions pour les quatre algorithmes d'ordonnancement, ce qui améliore la maintenabilité et réduit la redondance du code. Voici les fonctions principales recyclées :

## 2.1 Fonctions de gestion de liste chainée (Liste.c / Liste.h)

Les structures de liste chainée (struct celluleListe) sont utilisées pour stocker les processus dans tous les algorithmes. Les fonctions comme estVideListe(), inserProcListe(), trier_par_arrivee() sont appelées par les quatre algorithmes pour gérer la collection de processus, qu'il s'agisse de trier les processus par heure d'arrivée ou d'insérer de nouveaux processus dans la liste.

L’utilisation d’une liste chainée plutôt qu’une file est justifiée pour l’absence de complexité dans le code, sous prétexte que l’emfilage et le défilage ne sont pas beaucoup usé car la liste de processus est triée avant l’opération.

## 2.2 Fonctions de calcul de métriques (Calcul_temps.c / Calcul_temps.h)

Les fonctions contenues dans les fichiers calcul_temps sont universellement utilisées pour tous les algorithmes. Elles permettent de calculer les indicateurs de performance de l'ordonnancement, indépendamment de l'algorithme choisi. Ces fonctions sont appelées à la fin du cycle d'ordonnancement (dans ordonnancement.c) pour afficher les résultats pour FIFO, SJF, SJRF et RR.

## 2.3 Fonctions d'affichage (ordonnancement.c)

Les fonctions d'affichage sont partagées par les quatre algorithmes. Elles génèrent un tableau standardisé pour visualiser l'état des processus (prêt, en cours, bloqué, terminé) et les métriques calculées, ce qui assure une cohérence dans la présentation des résultats pour tous les ordonnancements.

## 2.4 Fonctions de gestion des processus (ProcStruct.c / ProcStruct.h)

Les fonctions de vérification d'état (est_pret(), est_en_cours(), est_bloque(), est_termine()) et d'initialisation (initProc(), creerCycle()) sont utilisées par chaque algorithme pour manipuler les processus et leurs cycles CPU/E/S. Par exemple, est_pret() est appelée dans chercher_premier_pret() (FIFO/RR), chercher_sjf() (SJF) et chercher_sjrf() (SJRF) pour identifier les processus disponibles.

## 2.5 Module de Lecture de Fichiers de Configuration (lecture_fichier.c/lecture_fichier.h)

Ce module dédié à la lecture de fichiers texte de processus répond à l'exigence du sujet de prise en charge de jeux d'essai conséquents sans modification du code source. La fonction principale `lire_processus()` est universellement réutilisée, quel que soit l'algorithme d'ordonnancement sélectionné par l'utilisateur.
Ce module implémente :

- un format de fichier structuré et lisible : `pid arrivee temps_CPU... / temps_ES...`, avec prise en charge des commentaires et des lignes vides ;
- la création automatique des cycles CPU et E/S pour chaque processus lu, en s'appuyant sur les fonctions d'initialisation du module ProcStruct ;
- la gestion des erreurs d'ouverture de fichier et de format invalide, avec des messages d'alerte explicites.

## 2.6 Choix des structures

### Processus

Un processus contient en général :

- Son temps d'arrivée dans le système
- Son numéro ou pid
- Un cycle CPU
- Un cycle E/S
- Son état, qui peut être prêt, en cours, bloqué, ou terminé. Cependant, des états

spéciaux sont rajoutés pour faire fonctionner l'algorithme de RR (état 6), et pour gérer l'affichage lorsque le processus est fini (état 5).
- Un temps de fin de processus pour la restitution
- Un temps d'entrée dans le premier burst CPU pour le calcul du temps d'attente
- Un temps total de burst CPU
- Un temps total de phase d'E/S
- Et un booléen pour verifier si le temps d'entrée dans le premier burst CPU a déjà été affecté.

# 3. Analyse détaillée du fichier ordonnancement.c

Le fichier ordonnancement.c est le cœur du simulateur : il centralise la logique d'ordonnancement et coordonne les appels aux fonctions spécifiques de chaque algorithme. Voici sa structure et ses fonctionnalités clés :

## 3.1 Définitions et constantes

Des constantes de codes couleurs ANSI sont définies pour améliorer la lisibilité de l'affichage (différenciation des états des processus : jaune pour "prêt", bleu pour "CPU", violet pour "bloqué", vert pour "terminé"). Les identifiants des algorithmes (ALGO_FIFO=0, ALGO_SJF=1, ALGO_SJRF=2, ALGO_RR=3) permettent de brancher la logique spécifique de chaque ordonnanceur.

## 3.2 Fonctions d'affichage

- afficher_entete() : Génère l'en-tête du tableau d'ordonnancement (colonne "Temps" + colonnes par PID de processus).
- afficher_ligne() : Affiche l'état de chaque processus à un instant temps donné, en utilisant les codes couleurs pour différencier les états.
- afficher_restitution(), afficher_attente(), afficher_reponse() : Affichent les métriques calculées pour chaque processus après l'ordonnancement.
- afficher_occupation() : Affiche le taux d'occupation du CPU pour l'ensemble du cycle.
- afficher_restitution_moyen(), afficher_attente_moyen(), afficher_reponse_moyen() : Affichent les métriques calculées pour tout l'ensemble du système durant l'ordonnancement.

## 3.3 Fonctions utilitaires

- chercher_premier_pret() : Renvoie le premier processus dans l'état "prêt" (utilisé par FIFO et RR, qui partagent la même logique de sélection du premier

processus disponible).

- trier_par_arrivee() : Trie la liste des processus par heure d'arrivée (appelée au début de l'ordonnancement pour tous les algorithmes, garantissant que les processus sont traités dans l'ordre d'arrivée lorsque nécessaire).
- tous_termines_file() : Vérifie si tous les processus sont terminés (condition d'arrêt de la boucle d'ordonnancement).

## 3.4 Fonction principale : ordonnancer()

C'est la fonction pivot du fichier, avec les étapes clés suivantes :

**Initialisation** : Tri de la liste des processus par arrivée, affichage de l'en-tête (nom de l'algorithme, quantum pour RR) et de la légende des états.

**Boucle d'ordonnancement** (tant que tous les processus ne sont pas terminés) :

- **Mise à jour des états** : Passage des processus "non arrivés" à "prêt" quand leur heure d'arrivée est atteinte ; décrémentation de la durée des cycles E/S pour les processus bloqués (puis passage à "prêt" si le cycle E/S est terminé).
- **Gestion du processus en cours** : Décrémentation du cycle CPU du processus en cours, mise à jour du compteur de quantum (pour RR), passage à "bloqué" (si E/S restantes) ou "terminé" (si plus de cycles) lorsque le cycle CPU est épuisé.
- **Préemption (SJRF)** : Si un processus plus court (en termes de cycle CPU restant) devient disponible, le processus en cours est remis à l'état "prêt" (spécificité de SJRF).
- **Sélection du prochain processus** : Selon l'algorithme choisi :
  1. FIFO/RR : chercher_premier_pret()
  2. SJF : chercher_sjf()
  3. SJRF : chercher_sjrf()
- **Affichage** : Appel à afficher_ligne() pour visualiser l'état des processus à l'instant temps.
- **Mise à jour du temps** : Incrémentation du compteur de temps, pause de 1 seconde (pour la lisibilité de la simulation).

**Affichage des métriques** : Après la boucle, appel des fonctions d'affichage des temps de restitution, attente, réponse et du taux d'occupation du CPU.

L’ordre mentionnée ici est primordiale et son suivi est très important pour que l’algorithme fonctionne.

### 3.5 Analyse détaillée du fichier main.c (Noyau de l'Interface en Ligne de Commande)

Le fichier main.c est le point d'entrée du programme, et il intègre l'intégralité de la logique de l'interface en ligne de commande. Sa structure est organisée en trois blocs fonctionnels clés :

**Fonction d'affichage de l'aide** : afficher_aide() Cette fonction génère une documentation utilisateur complète dans la console : elle présente la syntaxe d'utilisation du programme, détaille chaque paramètre disponible, et fournit des exemples de commandes concrets. Elle est accessible à tout moment via le paramètre `-h`, sans nécessiter le lancement d'une simulation.

**Fonction de création de la liste par défaut** : creer_liste_defaut() Cette fonction initialise une liste de processus de test préconfigurés, utilisée lorsque l'utilisateur ne spécifie pas de fichier de processus via le paramètre `-f`. Elle permet de lancer une simulation en une seule commande, sans préparation de fichier, pour des tests rapides des algorithmes.

**Fonction principale : main()**
C'est la fonction pivot de la gestion CLI, avec un déroulement structuré :

1. **Initialisation des paramètres** : définition des valeurs par défaut (quantum = 2, pas d'export CSV, liste par défaut) ;
2. **Parsing des arguments** : parcours de la liste des paramètres saisis par l'utilisateur, avec affectation des configurations correspondantes ;
3. **Vérifications de validité**: contrôle de la présence d'un algorithme spécifié, de la validité du quantum, de l'existence du fichier de processus si spécifié ;
4. **Chargement des processu** : appel à `lire_processus()` si un fichier est fourni, ou à `creer_liste_defaut()` sinon ;
5. **Lancement de la simulation** : appel à la fonction `ordonnancer()` avec les configurations validées, pour exécuter l'ordonnancement selon l'algorithme choisi.

# 4. Points forts du projet

## Points forts

**Réutilisation des fonctions** : La modularité du code permet de réutiliser des blocs fonctionnels pour les quatre algorithmes, réduisant la duplication et facilitant la maintenance.

**Lisibilité** : L'utilisation de codes couleurs et d'un format de tableau structuré rend la simulation facile à comprendre.

**Modularité** : Chaque algorithme a son propre fichier avec sa fonction de sélection de processus, ce qui sépare les logiques spécifiques et simplifie l'ajout de nouveaux algorithmes.

# 5. Conclusion

Ce projet met en œuvre quatre algorithmes d'ordonnancement de processus avec une forte emphasis sur la réutilisation des fonctions, ce qui assure la cohérence et la maintenabilité du code. Le fichier ordonnancement.c centralise la logique commune, tandis que les fichiers spécifiques à chaque algorithme implémentent les règles de sélection des processus. Le simulateur permet de visualiser et de mesurer les performances de chaque algorithme, ce qui en fait un outil pédagogique efficace pour comprendre le fonctionnement des ordonnanceurs de systèmes d'exploitation.